AI Usage Log : Sprint #2 Mamadou Camara

Task 2.1: Acceptance test examples

Purpose of AI Use: Brainstorming and proposing examples of acceptance tests

Chat Link or Prompt/Response:

Prompt: What's the best structure to adapt when making an acceptance test. Give me an example based on a job posting application.

Response: Acceptance tests are short, checkable scenarios that say when a user story counts as done. The usual format is Given / When / Then. Below is a GitHub issue template and example tests for the Sprint 2 features. I haven't seen your actual US numbers, so the IDs are placeholders.

Issue template

Title: AT-12: Upload resume with valid file (US-08)
Labels: acceptance-test, sprint-2, plus a feature label such as resume
Milestone: Sprint 2

User story: #8 (US-08)

Related tasks: #31, #32, #33

Scenario

Given I am a logged-in job seeker on the Resume page

When I upload a PDF smaller than 5 MB

Then the resume appears in my resume list with its file name and upload date

And I see a success message

Test data / preconditions: user test@demo.com, sample.pdf (1 MB)

Status: Not run / Pass / Fail

To link the issue, put Related to #8 in the body, and add the test to the user story's checklist (- [ ] #AT-12). You can also use GitHub's sub-issue feature.

Example acceptance tests (2 per story)

Registration and login (Sprint 1)

1. **Valid signup:** Given I am on the signup page, when I submit a unique email and a valid password, then my account is created and I'm redirected to the dashboard.

2. **Invalid login:** Given I have an account, when I log in with a wrong password, then I see "Invalid credentials" and no session is created.
3. **Duplicate email (optional extra):** Given the email already exists, when I sign up with it, then I see an error and no second account is created.

AI-Suggested Content: Generated an example of an acceptance test.

Validation: Compared the practices of the industry against the prompt answer and reviewed them during the team meeting.

Decision: Accepted but modified.

Reflection: AI helped identify a simple example.

Responsible Person: Mamadou Camara

Task 2.2: Creating setup/running files for the application

Purpose of AI Use: What are good examples of Continuous Integration Pipelines to implement for our project.

Chat Link or Prompt/Response:

Prompt: What would be an example of a Continuous Integration pipeline in the repository that would be beneficial for my project and how do I implement it in github.

Response:

A CI pipeline for your stack is one GitHub Actions workflow that runs on every push and pull request. The code lives in `Sprint 1/` and `Sprint 2/` subfolders, so the workflow below points at those paths. Your app is a FastAPI backend (Python) with a React 19 + TypeScript + Vite frontend, and the database is hosted on Supabase.

What the pipeline should do

Job	Checks	Why it helps you
Backend	Install `requirements.txt`, lint with `ruff`, run `pytest`	Catches broken endpoints (auth, resume, jobs, applications) before merge
Frontend	`npm ci`, `eslint`, `tsc` type-check, `vite build`	Catches TypeScript errors and a broken build
Secret scan (optional)	`gitleaks`	Your Definition of Done says "no secrets committed"

Two design decisions matter for your project:

- **Don't point CI at the shared Supabase database.** Tests that write to it are risky and flaky. Use a throwaway Postgres service container in the workflow instead. Your `database.py` requires a `postgresql+pg8000://` URL, and that works with a Postgres service.
- **Make your acceptance tests run in CI.** If you write them as pytest tests, such as `test_register_duplicate_email` or `test_upload_rejects_exe`, you can link each GitHub acceptance-test issue to the test that automates it.

AI-Suggested Content: Generated multiple examples that could be used.

Validation: Compared the common pipelines in github projects against the prompt answer and discussed it heavily during the team meeting.

Decision: Accepted but modified.

Reflection: AI helps give an adequate idea of potential pipelines quickly. However, has to be discussed within the team to find the most adequate ones.

Responsible Person: Mamadou Camara

Task 2.3: Commit description for the CI implementation

Purpose of AI Use: Proposing and creating a descriptive commit and pull request description that includes a summary, a setup note and a checklist of test plans, in the same format as the profile management pull request.

Chat Link or Prompt/Response:

Prompt: (pasted the profile management description, “Profile & Resume Management + Profile Photos + Retractable Sidebar”) Give me a similar commit description for this CI Implementation. Give me it in a markdown format

Answer:

Add CI pipeline and automated backend tests

* Added a GitHub Actions workflow (`.github/workflows/ci.yml`) that runs automatically on every push to `main` and every pull request into `main`, with a manual "Run workflow" button for demos

* **Backend job:** starts a throwaway PostgreSQL 16 container, installs dependencies on Python 3.13 and runs the pytest suite

* **Frontend job:** installs with `npm ci`, then runs ESLint and a production Vite build

* Added 56 automated tests in `Sprint 2/backend/tests/` covering Login/Registration (19), Resume Management (20) and Profile and photo (17)

* Shared fixtures in `conftest.py` (`client`, `user`, `make_user`) mean a new feature only needs a new `test_<feature>.py` file; the workflow does not change

* Tests refuse to run unless `DATABASE_URL` points at localhost, so they can never write to the shared Supabase database

* Uploaded files go to a temporary folder during tests, so `backend/uploads/` stays clean

* Added `requirements-dev.txt` (pytest, httpx) and `pytest.ini`; production dependencies are unchanged

* README now has a project overview paragraph and a "Testing and Continuous Integration" section covering local runs and how to add tests

* `.pytest_cache/` added to `.gitignore`

Setup note: to run the tests locally, start a disposable Postgres first (docker run -d --name cc-test-db -p 5432:5432 -e POSTGRES_PASSWORD=postgres -e POSTGRES_DB=careerconnect_test postgres:16), then run DATABASE_URL=postgresql+pg8000://postgres:postgres@localhost:5432/careerconnect_test pytest -v from Sprint 2/backend. No changes to the shared database are needed.

Test plan

- [x] Run `pytest -v` locally against a fresh Postgres: all 56 tests pass

- [x] Run pytest with a non-local or unset `DATABASE_URL`: it stops with a clear message and touches nothing

- [x] Parse `ci.yml` as YAML: valid, with the `backend` and `frontend` jobs

- [ ] Open a PR into `main`: both jobs start automatically and turn green

- [ ] Push a deliberately broken assertion: the backend job turns red and names the failing test, then green again after reverting

- [ ] Run `npm ci && npm run lint && npm run build` in `Sprint 2/frontend`: confirms the frontend job will pass

- [ ] In GitHub branch protection, mark `Backend tests (pytest)` and `Frontend (lint + build)` as required checks

AI-Suggested Content: Generated a markdown file containing the requested elements

Validation: Compared each bullet with the files that were actually added and with the test results, and ticked off the pending items as I completed them.

Decision: Accepted but modified to remove unnecessary information and the files changes.

Reflection: AI helped create a very good description of the commit and made the writing process faster. However, I modified it because it had more information than needed.

Responsible Person: Mamadou Camara

Task 2.4: Retractable sidebar with tabs

Purpose of AI Use: Coding and UI design: building a sidebar that slides between a full form and a slim icon-only form, and turning the dashboard into a shell with tabs.

Chat Link or Prompt/Response:

Prompt: Show an example of the process necessary to implement these features. I want to add a clean retractable side bar where we would add tabs in the future.

Answer:

1. Turn `Dashboard.tsx` into a thin shell. It holds the layout and the active tab, and each tab becomes its own component under `components/tabs/`:

tsx

```
// Dashboard.tsx — adding a future tab = adding one entry here
const TABS = [
  { id: "jobs",    label: "Job Dashboard", icon: <BriefcaseIcon />, render: () => <JobDashboardTab ... /> },
  { id: "profile", label: "Profile",       icon: <UserIcon />,     render: () => <ProfileTab ... /> },
];
```

2. Create `components/Sidebar.tsx`. It renders the `TABS` list, marks the active tab, and has a collapse button. The collapsed state is saved in `localStorage` under `cc_sidebar_collapsed`, so it survives a refresh.
3. Control the width in CSS with a grid variable, so the page content slides over smoothly:

css

```
.cc-shell { display: grid; grid-template-columns: var(--cc-side-w) 1fr; }
.cc-shell.is-collapsed { --cc-side-w: 64px; }
```

4. Move the resume logic out of the old `ResumeSection.tsx` into `components/ResumeManager.tsx`, which is shown inside the Profile tab. Job Dashboard stays the default tab, with the same content as before.
5. Add `ProfileTab.tsx` for job seekers and `RecruiterProfileTab.tsx` for recruiters, so both roles get the same tab structure.

AI-Suggested Content: The shell and tab architecture, the sidebar component and its CSS, the relocation of the resume manager into the Profile tab, and the avatar-to-Profile navigation.

Validation: Compared the old code with the suggestion and looked through react documentations.

Decision: Accepted but modified to fit my code logic.

Reflection: AI helped create a very template that fits the demanded process.

Responsible Person: Mamadou Camara

Task 2.5: Profile picture upload and storage

Purpose of AI Use: Let every user upload, view and remove a profile picture, with safe server-side storage, and fixing how it is cropped in the sidebar.

Chat Link or Prompt/Response:

Prompt: What's the best structure to handle profile picture selection/storing in our project.

Response:

Backend: add three endpoints in `routers/profile.py`:

- `GET /api/profile/photo` returns the image, or 204 if the user has none.
- `PUT /api/profile/photo` uploads or replaces it.
- `DELETE /api/profile/photo` removes it.

2. Storage: save files on disk as `backend/uploads/avatars/<user_id>.<ext>`. There is no database column and no schema change, and the user id is the file name, so each user has exactly one photo.
3. Safety checks on upload:
   - the file type is decided by reading the file's first bytes (JPG, PNG or WebP), not by trusting the extension or the content-type header;
   - maximum size is 2 MB;
   - SVG is rejected, because it can contain scripts;
   - responses carry `Cache-Control: no-store` and `X-Content-Type-Options: nosniff`.
4. Frontend, in `api.ts`: add `getProfilePhoto` (returns a blob URL, or `null` on 204), `uploadProfilePhoto` and `deleteProfilePhoto`.
5. Add `components/ProfilePhoto.tsx` (click, upload, remove, and initials as a fallback via `avatar.ts`). `Dashboard.tsx` owns the photo URL and refreshes it through `handlePhotoChange`, so the header and sidebar avatars update together.

AI-Suggested Content: Generated an example of a structure to store and display profile pictures..

Validation: Compared the practices of others against the prompt answer and reviewed them during the team meeting.

Decision: Accepted, but modified.

Reflection: AI added safety checks I would not have thought of, such as rejecting SVG and checking the actual file bytes.

Responsible Person: Mamadou Camara